Copenhagen Cupcakes

Artisan Cupcakes

Team A

Alexander

AlecDH

alha1004

Lukas

Ledut-dev

lued1000

Magnus

Magnus128

magu1000

Olga

Olga-Willadsen

olwi1000



12.11.2026

Indhold

Indledning 1
Baggrund 2
Teknologivalg 2
Krav 2
  Vision 3
  Funktionelle krav (user stories) 3
  Ikke-funktionelle krav 3
Domænemodel 4
ER diagram 4
Klassediagram 5
Navigationsdiagram (Bilag) 5
Implementering 6
    Session 6
    Særligt lækker kode 6
    Fejlhåndtering 7
Test 8
    Unit test 8
    Integrations Test 8
Status på implementering 9
Proces 9
    Gitstrategi 9
    Kodestandard 9
    Roller 10
Bilag 11

Indledning

Denne rapport beskriver et systemudviklingsprojekt på 2. semester af datamatiker-uddannelsen på Erhvervsakademi København med tilhørende prøveeksamen. Rapportens antagne læser er en fagfælle, i.e., en datamatiker-studerende på 2. semester.

Der skal udvikles et fullstack-system til en cupcake-butik. Systemet skal både fungere for vores kunde som driftsredskab og for deres kunder som bestillingsplatform. Opgaven stiller krav om database som overholder 3.normalform, domænemodel og navigationsdiagram.

Baggrund

Copenhagen Cupcakes er en mindre virksomhed baseret i København, der sælger cupcakes. Copenhagen Cupcakes skal bruge et system, som dels kan bruges af deres kunder til at oprette bestillinger, men også kan bruges af ansatte til at holde styr på aktive såvel som arkiverede bestillinger. Copenhagen Cupcakes har ikke et fast menukort. Kunden bestemmer derimod selv kombinationen af kagebunden og den topping der kommer på. Kunden skal kunne vælge mellem alle udbudte smagsvarianter af henholdsvis bund og topping.
De ansatte skal have personlige profiler, de skal kunne se aktive bestillinger i systemet, godkende dem til produktion og kunne følge med i salgsstatistik. Både kunder og ansatte skal kunne logge ind via samme portal men have forskellige funktioner til rådighed på deres profiler.

Teknologivalg

IntelliJ 2025.3.2 (2026.2.4)

Java 21

Javalin 7.2.3

JDBC 4.3

HikariCP 7.0.2

Postgresql 42.7.8

pgAdmin 4.9.18

HTML

CSS

Thymeleaf

Krav

Vision

En butik, der sælger cupcakes, hvor kunderne selv kan vælge kombinationen af bund og topping, ønsker en hjemmeside. Der ønskes et login-system, der gør kunderne i stand til at sammensætte, bestille og betale cupcakes til senere afhentning, og administratorer i stand til at følge op på bestillinger.

Funktionelle krav (user stories)

US-1: Som kunde kan jeg bestille og betale cupcakes med en valgfri bund og top, sådan at jeg senere kan køre forbi butikken og hente min ordre uden at stå i kø.

US-2: Som kunde kan jeg oprette en konto/profil for at kunne betale og gemme en ordre.

US-3: Som kunde kan jeg se mine valgte ordrelinjer i en indkøbskurv, så jeg kan se den samlede pris.

US-4: Som kunde eller administrator kan jeg logge på systemet med e-mail og kodeord. Når jeg er logget på, skal jeg kunne se min e-mail på hver side (evt. i topmenuen, som vist på mockup'en).

US-5: Som administrator kan jeg se alle ordrer i systemet, så jeg kan se hvad der er blevet bestilt.

US-6: Som administrator kan jeg se alle kunder i systemet og deres ordrer, sådan at jeg kan følge op på ordrer og holde styr på mine kunder.

US-7: Som kunde kan jeg fjerne en ordrelinje fra min indkøbskurv, så jeg kan justere min ordre.

US-8: Som administrator kan jeg fjerne en ordre, så systemet ikke kommer til at indeholde ugyldige ordrer. F.eks. hvis kunden aldrig har betalt.

Til user story 4 har Product Owner besluttet, at vi bruger fornavn på vores kunder. Copenhagen Cupcakes er en lille butik, som gerne vil have et personligt forhold til de personer, der besøger dem. At bruge deres fornavn på hjemmesiden, når de kontakter kunderne eller overleverer bestillinger passer meget bedre til brandet. Samtidigt opfylder det ønsket om, at en bruger skal kunne se, at de er logget på. Derfor kræver vi også et fornavn, når kunderne opretter deres profil.

Vi havde i gruppen også diskussion med vores PO vedr. user stories 5 og 6 i forhold til at kunne definere, hvad der menes med "Så jeg kan se hvad der er bestilt" i US-5 og "Så jeg kan følge op på ordrer" i US-6.
Her endte vi med at blive enige om, at US-6 specifikt efterspørger en oversigt over aktive ordrer, som Copenhagen Cupcakes medarbejdere skal tage stilling til - f.eks. godkende/afvise ordrer, hvorimod US-5 efterspørger en generel ordrehistorik.

Ikke-funktionelle krav

- Beløb indsættes på en kundes konto direkte i Postgres databasen via PgAdmin, så en kunde kan betale for sine ordrer.
- Ordrer, kunder og øvrige data skal gemmes i en database.
- Databasen skal normaliseres på 3. normalform medmindre andet giver bedre mening.
- Kildekoden skal deles på GitHub.
- Det færdige produkt skal udvikles i Java, Javalin, Thymeleaf template engine, Postgres Database, HTML og CSS.
- Websitet skal helst kunne fungere tilfredsstillende både på en almindelig skærm og på en mobiltelefon (iPhone 15 og lignende).

Domænemodel

Vores domænemodel er generelt ret simpel, da det er ret begrænset hvor mange "objekter", der indgår i cupcakes og en butik. Vi havde dog fokus på, at vores kunde udmærker sig ved, at deres cupcakes opbygges modulært med en separat bund og top – derfor har vi valgt at fremhæve, at en Cupcake har en CupcakeTop og en CupcakeBottom.

ER diagram

Tabellerne cupcake_tops og cupcake_bottoms benytter deres kolonner flavor som primary key, da denne skal være unik. Hvis flavor ændres skyldes det at sortimentet i butikken ændres.

Copenhagen Cupcakes unikke salgsargument er kundens frie valg af kagebund og topping. Hver solgte cupcake kombinerer data fra de to tabeller og der opstår en en-til-mange relation mellem orderlines og hhv. Cupcake_tops og cupcake_bottoms, hvilket er mere effektivt og fleksibelt end en tabel med alle de mulige cupcakes.

Tabellerne order_history og orderline_history holder på butikkens statistik. Idet en bestilling godkendes i butikken, gemmes alle informationer om order og orderline her i tabellen. Tabellerne har ingen fremmednøgler til order, orderline, cupcake_tops eller cupcake_bottoms, således at informationerne for allerede udførte bestillinger ikke ændres, hvis butikkens udvalg eller priser gør. Dette understøtter brugen af flavor som primary key i cupcake_tops og cupcake_bottoms.

Selvom admins og customers begge er brugere i samme system, er de skilt ad i databasen, fordi fordelingen helst skulle være, at der er mange customers og kun en håndfuld admins. Vi diskuterede om typen af bruger skulle defineres med en Enum eller boolean attribut såsom isAdmin, men så ville vi få en kolonne, hvor langt de fleste rækker havde samme værdi.

Mens vi løbende har kigget tilbage på vores ERD i gruppen, har vi overvejet tabellen order_history. Vi startede med at give den en "id serial", som ikke matcher ordertabellens id. Vi blev dog enige om, at det ville være bedst at gemme en ordre i order_history med samme ordre_id som den havde i orders, da det udelukker alt forvirring eller eventuelle udfordringer i fremtiden vedr. ordre_id. Hvis vi havde mere tid til projektet ville vi have implementeret dette, dog endte vi ikke med at nå at få det med.

Vi har meget specifikt tilføjet ON DELETE CASCADE til orderlines-tabellen, da vi vil sørge for at alle relevante orderlines automatisk fjernes, når vi fjerner en ordre fra orders-tabellen.

```sql
-- 6. ORDERLINES
CREATE TABLE orderlines (
    orderline_id SERIAL PRIMARY KEY,
    order_id INTEGER REFERENCES orders(order_id) ON DELETE CASCADE,
    cupcake_topping VARCHAR REFERENCES cupcake_tops(flavor),
    cupcake_bottom VARCHAR REFERENCES cupcake_bottoms(flavor),
    quantity INTEGER
);
```

[1]

Vi har også valgt at have en UNIQUE-constraint på email i både admins og customers, da der kun må høre en enkel bruger til en specifik email-adresse. Vi har dog valgt IKKE at sætte email som primary key, da det kan ske, at en bruger ønsker at ændre deres email.

Klassediagram

De første redskaber vi brugte, var Domænemodel og ERD til design af systemet. Vi besluttede tidligt også at oprette et klassediagram til intern brug, dokumentation og fælles forståelse. I første omgang betød det, at vi satte os ned sammen og oprettede vores klassediagram v1 (se bilag 5) med udgangspunkt i diskussion af user stories og non-funktionelle krav. Det gav os et bedre samlet overblik over projektets struktur, nødvendige metoder, returværdier, og attribut-/metodenavne. Det gavnede os meget, da vi endelig begyndte på vores individuelle kodning af vertical slices (vi valgte generelt at definere hver User Stor som sin egen "slice"), da vi sammen havde en fælles forståelse for systemet.

I løbet af de sidste dage af projektet, har vi opdateret vores klassediagram (se bilag 6). Denne opdaterede version har vi oprettet for at have dokumentation på hele systemet, i tilfælde af at projektet "i virkeligheden" skulle overdrages til anden udvikler, eller at vi en gang i fremtiden skulle vende tilbage til det.

[1] Se SQL-init.sql, linje 50-55

Navigationsdiagram (Bilag)

Bilag 7 navigationsdiagram repræsenterer vores ambitioner for systemet.
Vi anvender en fælles header/navigations bar som indeholder link til login, link til kurv, samt "velkomst" besked/navn så en bruger kan se om de er logget ind. Hvis man er logget ind, erstattes login linket med logout.
Alt navigationen i denne fælles header er placeret der, da der skal være mulighed for at logge ind/ud, samt tilgå kurv fra alle generelle og "customer" sider. Samtidig opfylder velkomstbeskeden US-4, da en bruger altid vil kunne se deres navn, hvis de er logget ind.
Admin siderne er lidt specielle, da en admin ikke har en tilknyttet kurv, så de har, i headeren, kun mulighed for at logge ind/ud.

Implementering

Session

Systemet gemmer i session: currentUser, cupcakeTops, cupcakeBottoms og cart.

'currentUser' er en instans af klassen User og bliver defineret idet en bruger logger ind i systemet. User er en abstrakt superklasse som nedarves af både Customer og Admin. Logges der ind som customer sendes brugeren til bestillingsformularen og logges der ind som admin sendes brugeren til admin forsiden.

'cart' er en instans af klassen Order. Denne klasse holder på en liste af Orderline-objekter. Idet der trykkes "læg i kurv" tjekkes der for følgende: findes der allerede en 'currentUser' i sessionen, og findes der allerede en 'cart' i sessionen? Hvis der ikke findes en 'cart', bliver et nyt Order-objekt med status "CART" oprettet.
Findes der en 'cart' i sessionen og en 'currentUser' bliver user defineret som attribut i 'cart'. Hvis der findes en 'cart' i session, bliver der oprettet et Orderline-objekt som tilføjes til 'cart's liste af ordrelinjer.

Når der trykkes "bestil" på en kurv ændres status for kurven til "PLACED", og session-attributten 'cart' sættes til null, således at der nu ikke er en 'cart'.

Listerne cupcakeTops og cupcakeBottoms består af alle muligheder for hhv. topping og kagebund. Mulighederne skal bruges i bestillingsformularen. De gemmes i session, fordi denne HTML forventeligt vil blive indlæst flere gange, og på denne måde undgår systemet, at skulle hente de samme to lister fra databasen mere end én gang.

Særligt lækker kode

Grundet vores databasedesign vedr. Customers og Admins, har vi anvendt vores nedarvning fra User-superklassen til at håndtere validering om en bruger er en Admin.

Vi håndterer valideringen ved at lade en boolean som sessionAttribute returnere true hvis user er en instans af klasssen Admin.
Først forsøger vi at hente brugeren fra customers tabellen – hvis de findes her, oprettes der et Customer-objekt.

```
public User getUserByEmail(String email) throws DatabaseException {
    User user = null;
    String query = """
    SELECT email, password, name, customer_id, balance
    FROM customers
    WHERE email = ?""";
```

[2]

```
while (rs.next()) {
    String password = rs.getString( columnLabel: "password");
    String name = rs.getString( columnLabel: "name");
    int id = rs.getInt( columnLabel: "customer_id");
    double balance = rs.getDouble( columnLabel: "balance");
    user = new Customer(id, email, password, name, balance);
}
```

[3]

Hvis de ikke findes i customers, tjekkes der efterfølgende admins tabellen – hvis de findes her, oprettes der et Admin-objekt. Hele dette tjek sker i samme metode: getUserByEmail().

```
query = """
        SELECT email, password, name, admin_id
        FROM admins
        WHERE email = ?""";
```

[4]

```
while (rs.next()) {
    String password = rs.getString( columnLabel: "password");
    String name = rs.getString( columnLabel: "name");
    int id = rs.getInt( columnLabel: "admin_id");
    user = new Admin(id, email, password, name);
}
```

[5]

Den forskel i oprettelse anvender vi via metoden isUserAdmin(User user) i UserService. Her tjekker den hvilken klasse User er, og returnerer true/false, alt efter om en User er en instans af Admin.

```
public boolean isUserAdmin(User user){
    boolean isAdmin = false;

    if (user.getClass().getSimpleName().equalsIgnoreCase( anotherString: "admin")) {
        isAdmin = true;
    }
    return isAdmin;
}
```

[6]

Returværdien af denne isUserAdmin() bruger vi til at sætte sessionAttribute 'isUserAdmin' i vores UserController.

```
ctx.sessionAttribute("isUserAdmin", userService.isUserAdmin(user));
```

[7]

[2] Se UserMapper, linje 38-43
[3] Se UserMapper, linje 52-57
[4] Se UserMapper, linje 64-67
[5] Se UserMapper, linje 75-79
[6] Se UserService, linje 52-59
[7] Se UserController, linje 56

Fejlhåndtering

Vi har lavet nogle custom exceptions, som vi bruger til at tjekke, om der er sket en fejl og til at give den rigtige fejlbesked. Se f.eks. i UserController linje 65-80 vores register() metode. Her tjekkes først, om den indtastede email er tilgængelig, og hvis den ikke er, kastes en EmailNotAvailableException. Herefter tjekkes passwordets validitet, hvor PasswordNotValidException på samme måde kan kastes. I catch-blokken gemmes exceptionens getMessage() i et 'error'-javalin-attribut. I register.html (linje 29) bruges et Thymeleaf if-statement til at tjekke om 'error'-attributtet eksisterer, og hvis det gør, skrives teksten på siden.

Test

Unit test

Der er foretaget unit test på de metoder der laver beregninger på ordrer:

Order.getSize()

Order.getTotal()

Orderline.getTotalPrice()

Orderline.getIndividualPrice()

Unit testene er udført da user story 1-6 er reviewet og merget.

Integrations Test

Integrations test er påbegyndt og tester forbindelsen til databasen og mapper metoderne:

OrderMapper.getAllCakeTops()

OrderMapper.getOrderByid()

Vi har i løbet af projektet haft diskuteret både unit test og integrationstest, og ville som udgangspunkt gerne teste løbende.
I løbet af projektet havde vi svært ved at forstå hvad vi kunne teste da systemet hovedsageligt er transport af data.

Vi brugte vejledning med Tine til at tale igennem forskellige muligheder for test, og fik bekræftet at der ikke er meget at unit teste i dette system samt at der findes typer af integrationstest specifikt til frontend – f.eks. test af endpoints.

Status på implementering

Vi blev færdige med US: 1-6.
Vores gruppes valg ift. hvordan vi definerede US-6, her at en administrator har muligheden for at "afvise" en ordre, lægger sig meget tæt op ad US-8. Med en smule ekstra tid ville vi ret hurtigt kunne have implementeret dette.

Funktioner vi ikke har nået, men som giver mening for systemet:

- At oprette en ny administrator.
- At begrænse hvor mange ordrer, der kan hentes fra databasen på én gang.
- At vise ordrehistorikken over flere sider i stedet for at skulle render alle ordrerne på én gang.
- Hvis en kunde logger ud, mens de har noget i deres kurv, gemmes deres kurv til databasen
- Når en kunde logger ind, hentes deres gemte kurv fra databasen

Link til videodemonstration: Video demo på youtube

Proces

Gitstrategi

Vi brugte GitHub som processtyringsværktøj.

Vi lavede et issue for hver user story, og gav hver af dem et antal sub-issues. Vi gav main-branchen branch protection, og hvert gruppemedlem lavede en dev-branch. Vi lavede pull requests, når vi var færdige med en user story. Vi rørte ikke ved hinandens dev-branches bortset fra et par gange, hvor en pull request var tæt på at kunne blive merget.

Vi havde ikke de store konflikter med merging, men til fremtiden vil vi være bedre til at opdele store user stories i mindre dele og lave flere pull requests. Fremover vil vi også gå mere struktureret til løbende at evaluere gammel kode og oprette issues for at fixe fejl, der sniger sig gennem kode-review.

Kodestandard

Vi har, for det meste, genbrugt kodestandarden fra lifehack-projektet. Det er ikke alle vores standarder, vi har fulgt, f.eks. har vi ikke lavet relevante unit test før hvert pull request. Det har dog været rart at aftale projektstrukturen og navngivningsmønstre på forhånd, og vi lagde løbende mærke til, at vi ville have gavn af at aftale, om vi skrev kommentarer på dansk eller engelsk.

Roller

Rollerne fordelte vi sådan, at Product Owner og Tech Lead varede begge uger – vi håbede på at være færdige med at kode tidligt i uge to, og så ville de to roller være mere eller mindre unødvendige.

Vi oprettede en ny rolle som Notar. Denne og Scrum Master byttede ved udgangen af første uge.

Notarens rolle var at tage noter til alle interne diskussioner, så vi kunne holde styr på, hvad der var blevet besluttet og hvorfor.

Product Owner

Olga

Rollen som Product Owner var en udfordring at tage på mig, fordi den i sin natur laver en skævvridning af magten i gruppen, som ellers skulle være et ligeværdigt samarbejde.

Desuden kræver rollen som PO en anden indsigt i kundens ønsker, end resten af gruppen har, og det havde jeg naturligvis ikke, så mit grundlag for at træffe beslutninger på vegne af kunden, var ikke bedre end resten af gruppens.

I diskussioner om forskellige løsninger kunne der da også blive joket "det beslutter PO", men jeg mener, at vi som gruppe var gode til at tage hensyn til den manglende viden som rollen egentlig bygger på.

Tech Lead

Alexander

Vores kodestandard kopierede vi i høj grad fra vores lifehack-projekt med enkelte ændringer baseret på erfaringer fra dette. Diskussionen omkring disse ændringer var fælles og det samme var beslutninger såsom anvendelse af Date eller LocalDate. Som Tech Lead havde jeg intention om at være den primære reviewer ved pull requests, men i praksis blev pull requests reviewet af hvem end, der først havde tid.

Scrum Master / Notar

Lukas

Jeg var Scrum Master i vores første uge af projektet.
Som Scrum Master havde jeg meget fokus på at prøve at sikre at gruppen fik opsat mange "aftaler" og arbejdsgange som muligt, f.eks. Code Standard, hvordan PR ønsket at bruges, branch protection på Main i github, udarbejdning af acceptkriterier, hvordan vi tænker at håndterer arbejdsfordeling (f.eks. Havde vi stor fokus på at følge vertical slicing). Specielt i starten af projektet var der en stor mængde af disse aftaler og arbejdsgange der skulle opsættes.
Det betød også at rollen som Scrum Master ændrede sig lidt over tid, da det gik mere over til at sikre at gruppen holder fast i aftaler (mødetider, eventuel sygdom), samt at få sat konkrete opgaver i gang hvis gruppen "gik i stå".

I anden uge var notar, men da vi generelt var færdige med at kode hoveddelen af projektet, omhandlede det mere at notere spørgsmål vi havde til vejledning, eller hvis der skulle følges op på et møde.

Magnus

Jeg var notar i første uge og scrum-master i anden uge. Som notar fik jeg skrevet en masse noter den første dag, men siden har jeg næsten ikke skrevet noget. Jeg har svært ved at vurdere, hvor værdifulde mine noter er, men jeg tror processen i at skrive dem hjalp med at holde overblik. Som scrum-master prøvede jeg at minde min gruppe om at bruge kanban-board og pull requests.

Bilag

Bilag 1 – Domænemodel v1

Bilag 2 – Domænemodel v2

Bilag 3 – ERD v1

Bilag 4 – ERD v2

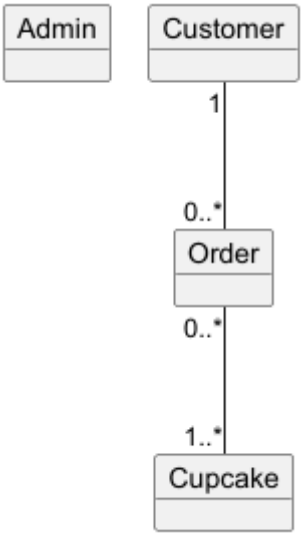
Bilag 5 – Klassediagram v1

Bilag 6 – Klassediagram v2

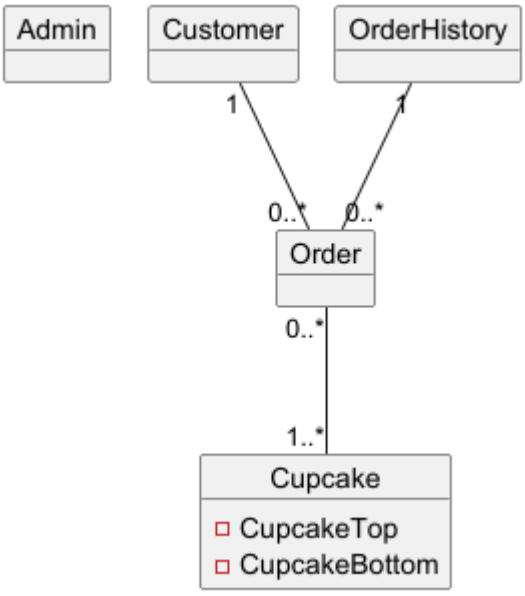
Bilag 7 – Navigationsdiagram

Bilag 8 – Prototype

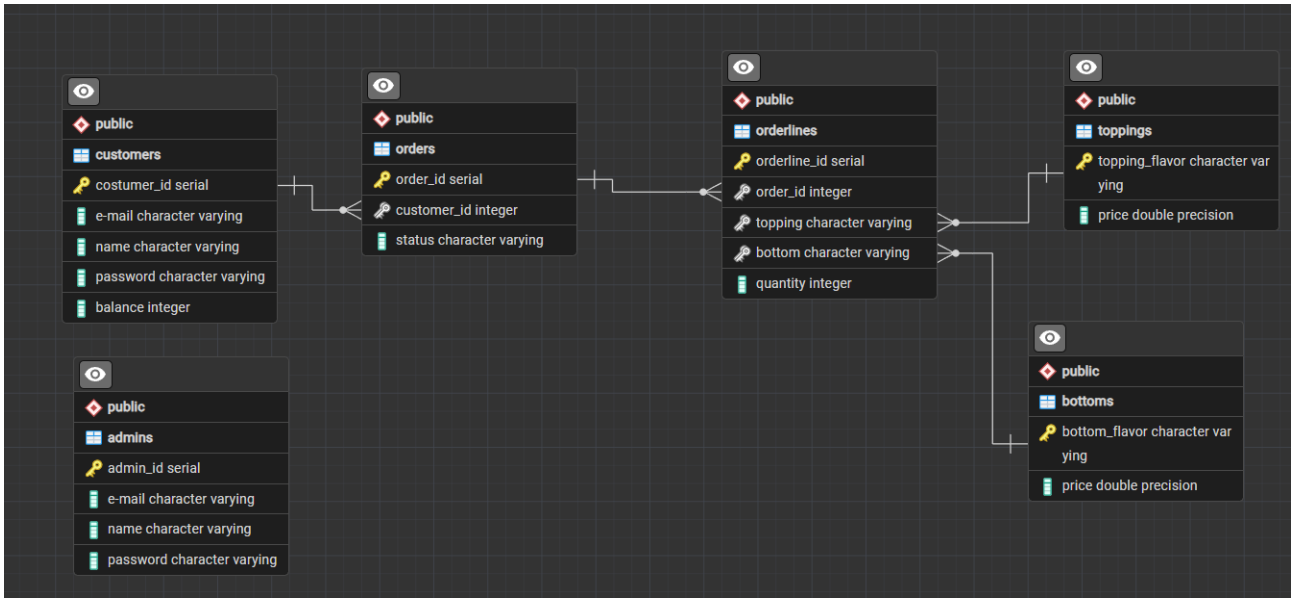
Bilag 1 - Domænemodel v1



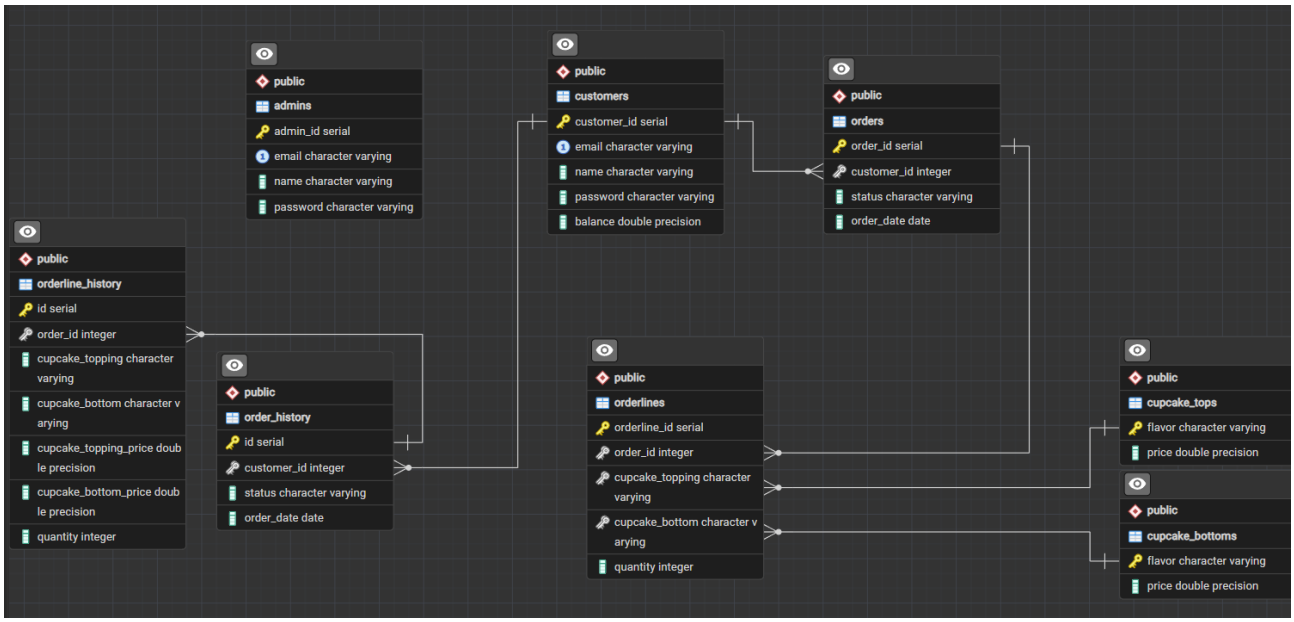
Bilag 2- Domænemodel v2



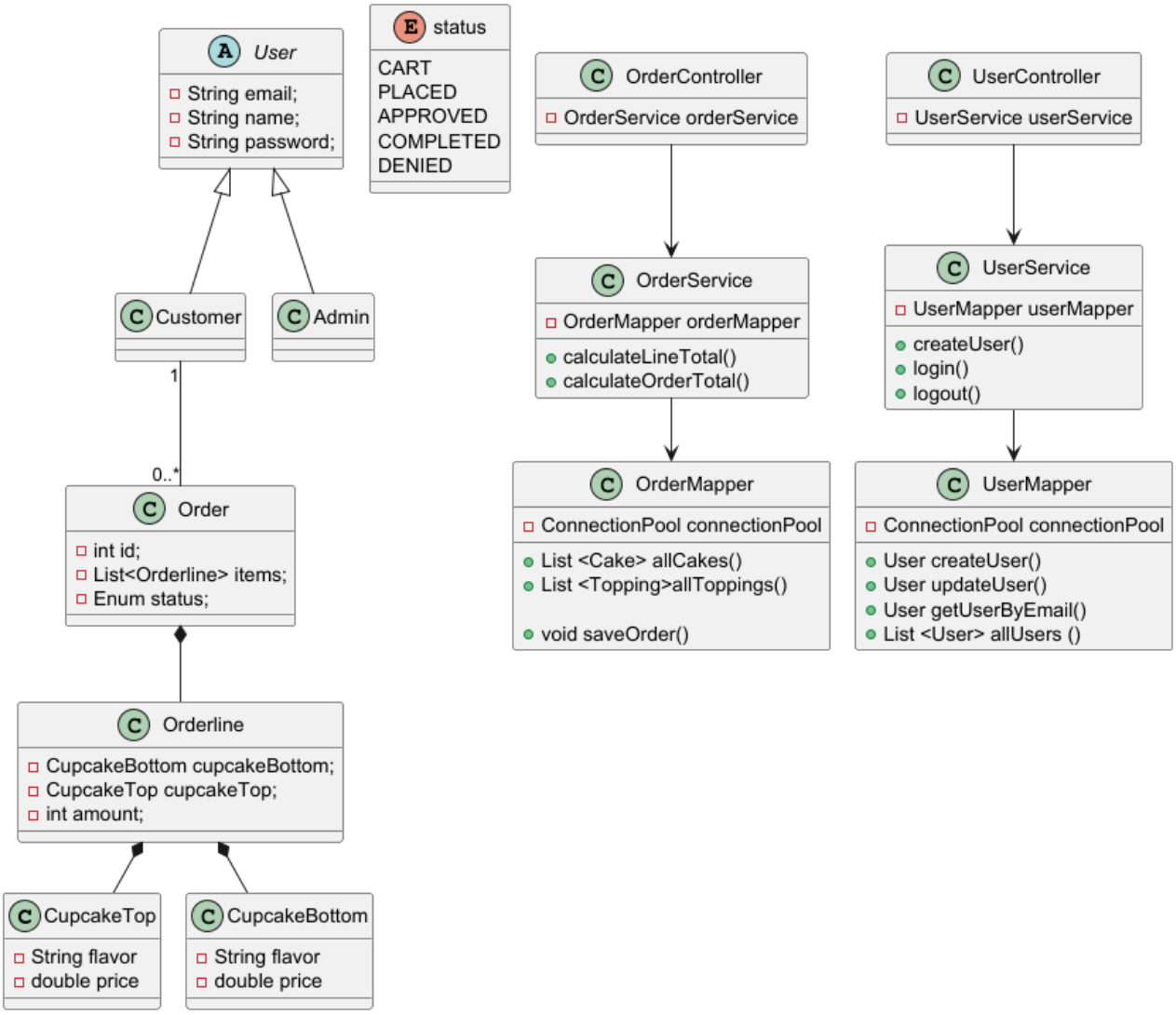
Bilag 3- ERD v1



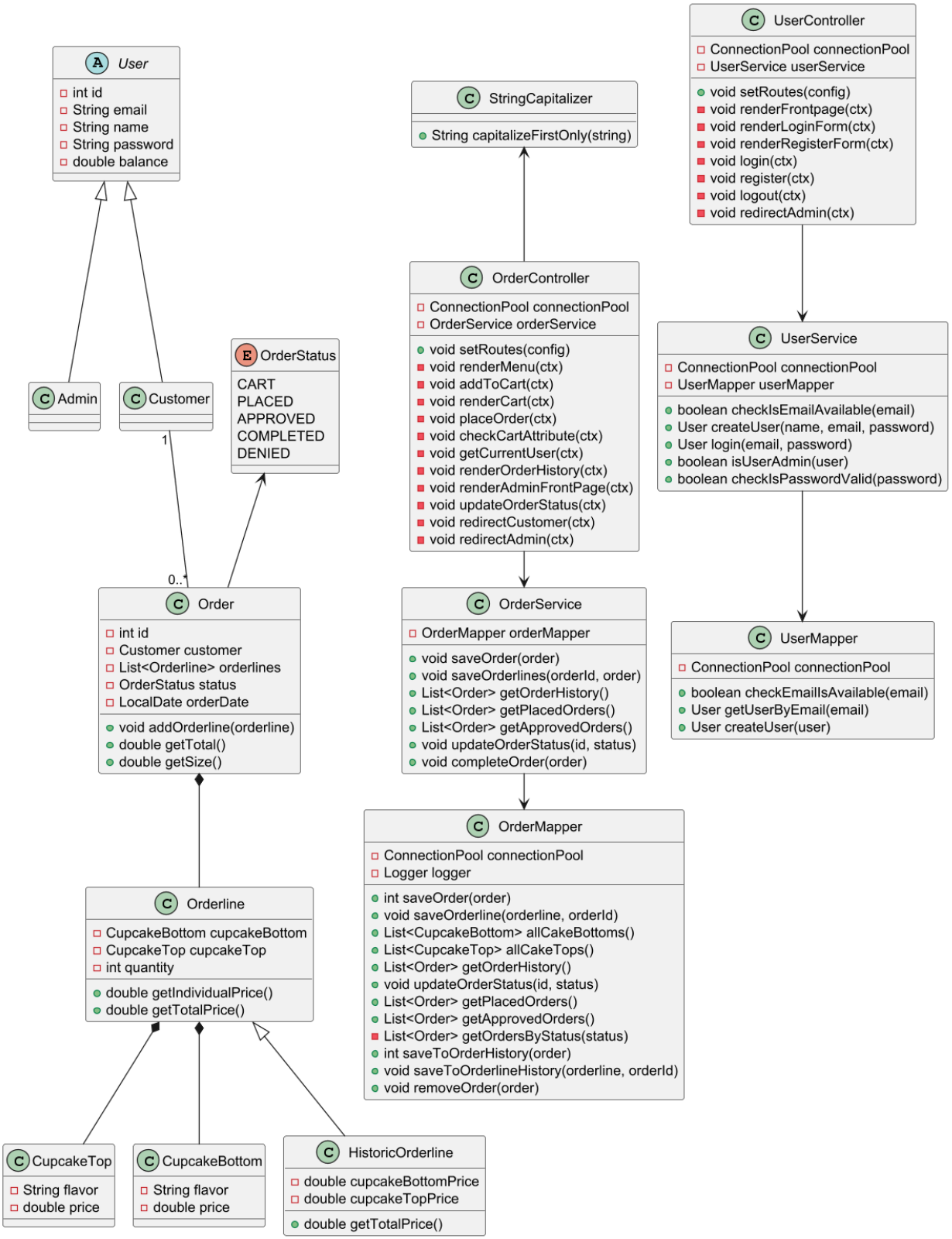
Bilag 4 - ERD v2



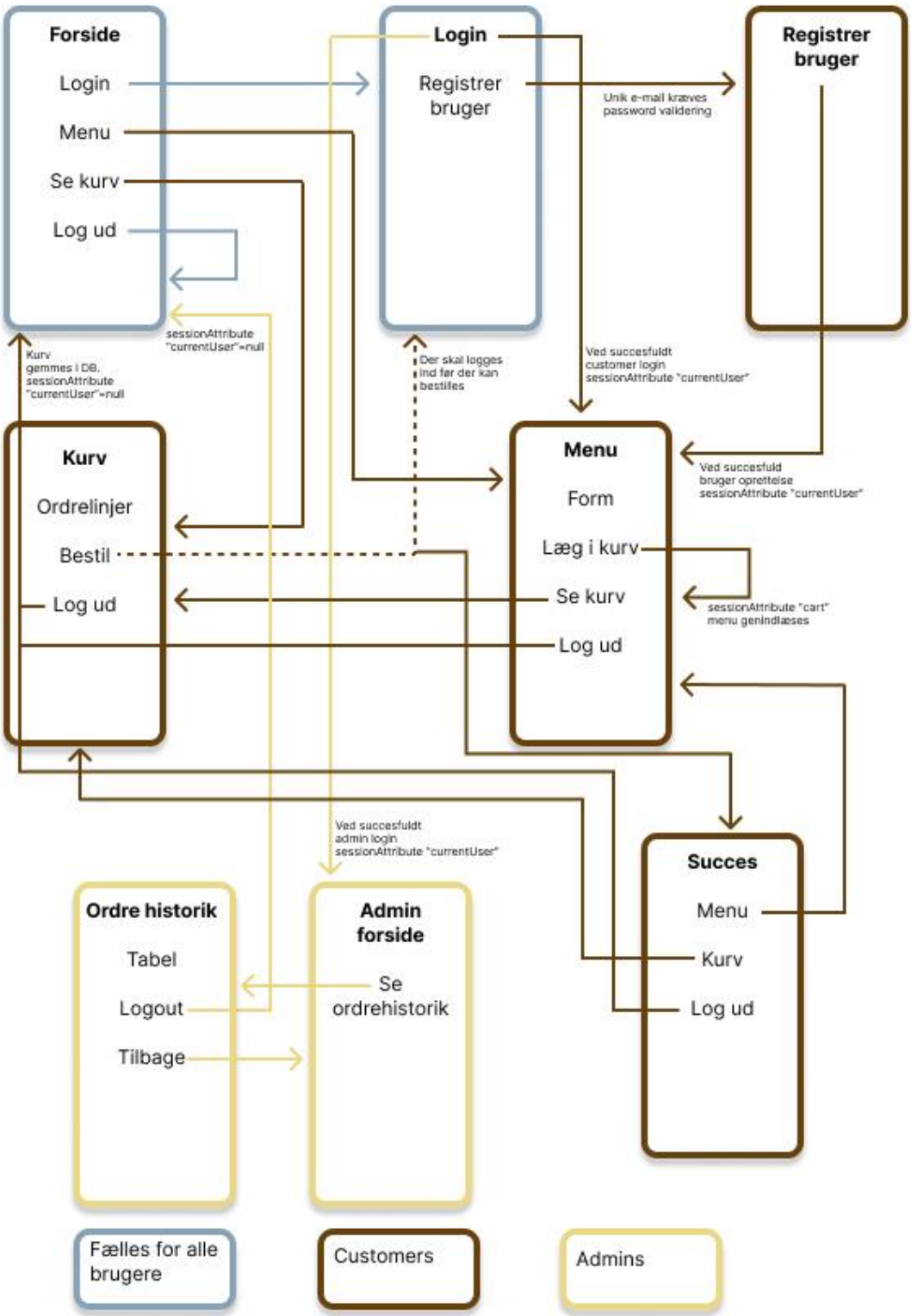
Bilag 5 - Klassediagram v1



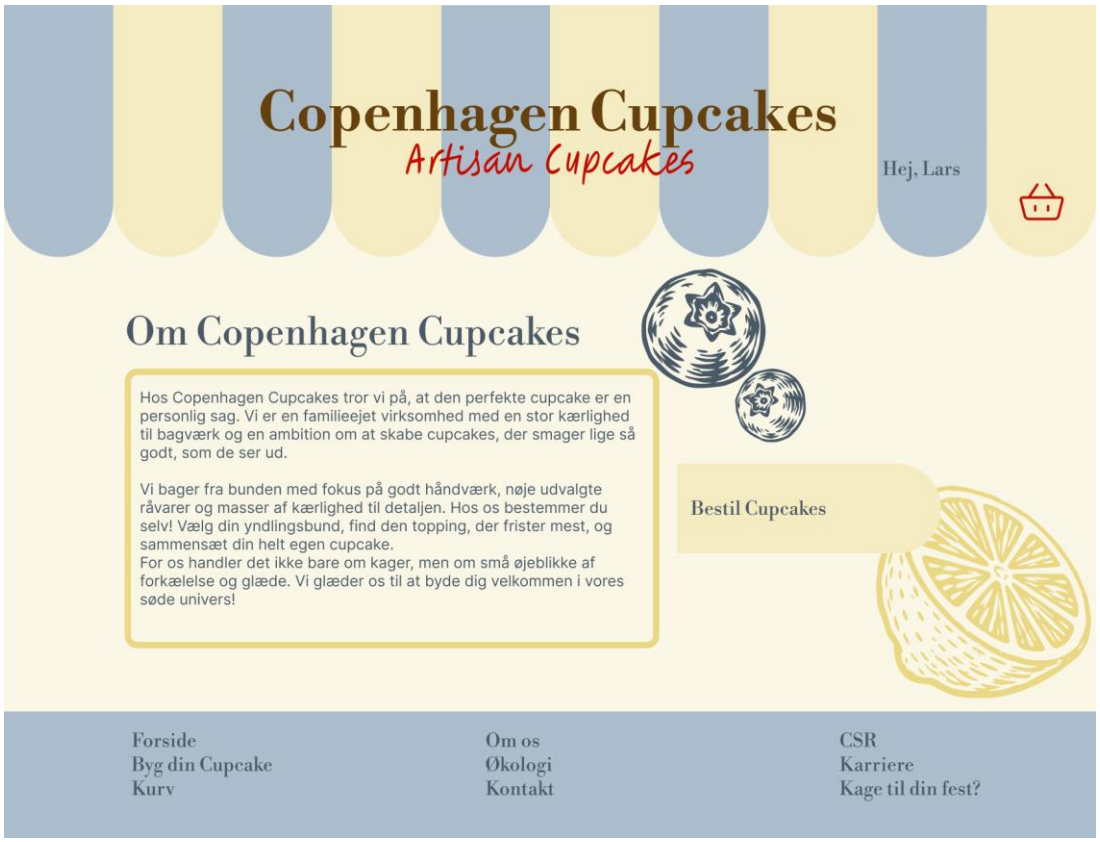
Bilag 6 - Klassediagram v2



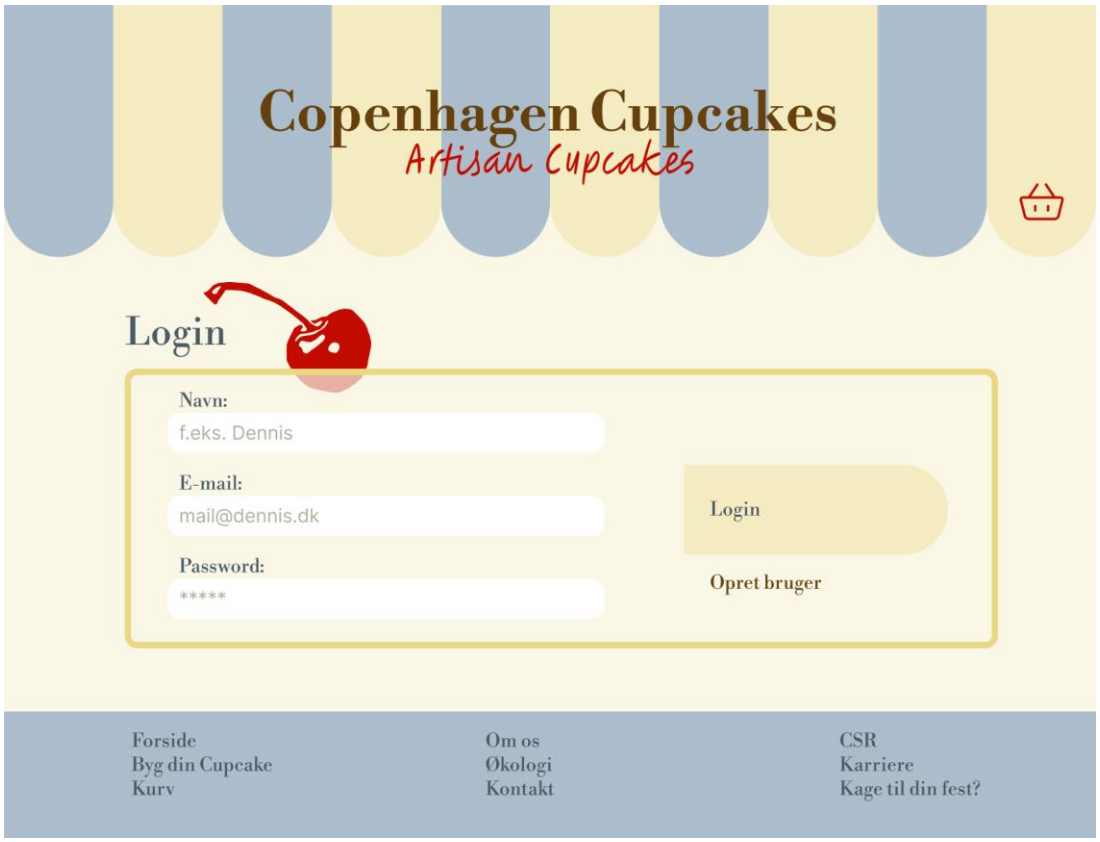
Bilag 7 – navigationsdiagram

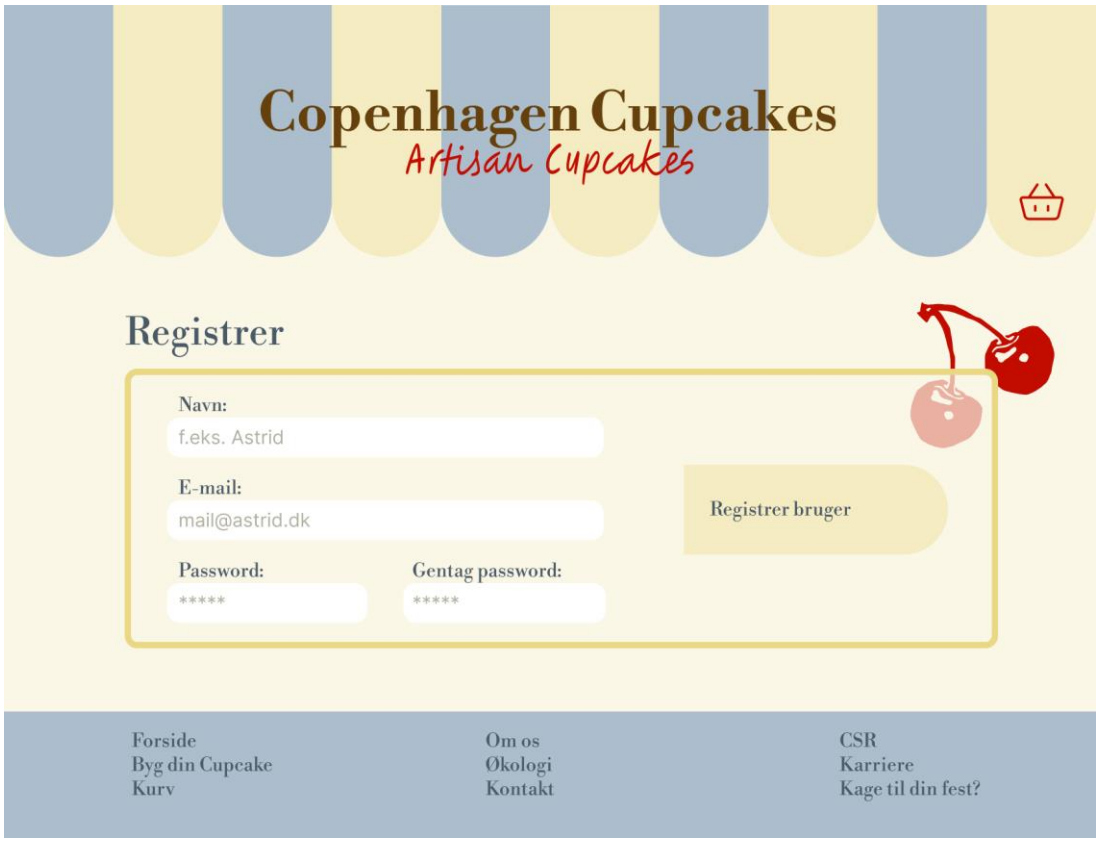

Forside
Login
Menu
Se kurv
Log ud
Login
Registrer bruger
Registrer bruger
Unik e-mail kræves
password validering
sessionAttribute "currentUser"=null
Kurv gemmes i DB. sessionAttribute "currentUser"=null
Der skal lægges ind før der kan bestilles
Ved succesfuldt customer login sessionAttribute "currentUser"
Kurv
Ordrelinjer
Bestil
Log ud
Menu
Form
Læg i kurv
Se kurv
Log ud
Ved succesfuld bruger oprettelse sessionAttribute "currentUser"
sessionAttribute "cart" menu genindlæses
Ved succesfuldt admin login sessionAttribute "currentUser"
Succes
Menu
Kurv
Log ud
Ordre historik
Tabel
Logout
Tilbage
Admin forside
Se ordrehistorik
Fælles for alle brugere
Customers
Admins

Bilag 8 - prototype





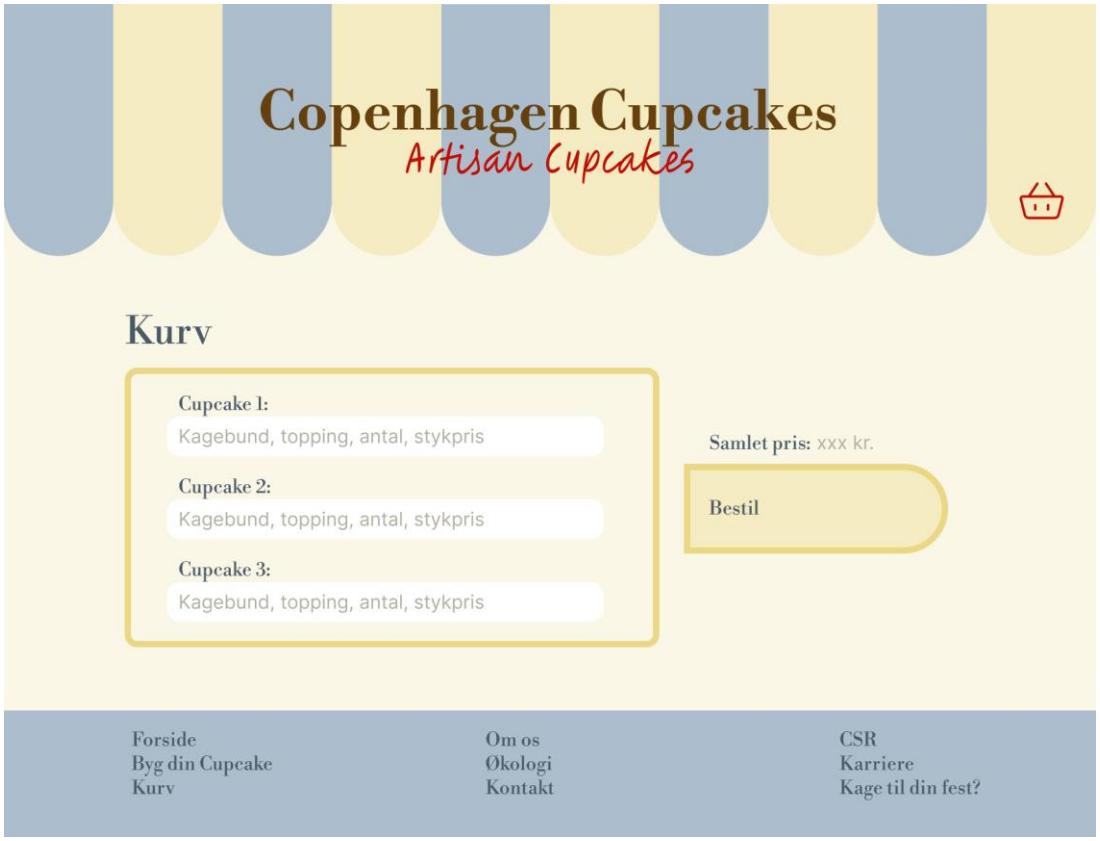

Copenhagen Cupcakes
Artisan Cupcakes
Login
Navn:
f.eks. Dennis
E-mail:
mail@dennis.dk
Password:
*****
Login
Opret bruger
Forside
Byg din Cupcake
Kurv
Om os
Økologi
Kontakt
CSR
Karriere
Kage til din fest?


Copenhagen Cupcakes
Artisan Cupcakes
Registrer
Navn:
f.eks. Astrid
E-mail:
mail@astrid.dk
Password:
*****
Gentag password:
*****
Registrer bruger
Forside
Byg din Cupcake
Kurv
Om os
Økologi
Kontakt
CSR
Karriere
Kage til din fest?





Her ses eksempler på hvordan systemet er designet til at være responsivt. Elementerne skulle på en smallere skærm flytte sig under hinanden.





De markerede bokse på 'kurv' og 'succes' er designet til at have overflow-x: hidden, sådan at det overordnede layout ikke afviger, men samtidig gav plads til mere end én ordrelinje.